UNIVERSIDADE FEDERAL DE PERNAMBUCO

Centro de Informática

Programa de Pós-Graduação em Desenvolvimento de Software Automotivo

Testing document

Start-Stop System

Group 3:

Andrei Rocha, Bruno Almeida, Gabriel Xavier,

Luana Maria, Matheus Lourenço

Date: April 24, 2026

1 Introduction

This document presents the testing strategy and results for the Start-Stop System developed in the context of the Automotive Software Development program. The objective of this document is to demonstrate that the system requirements are correctly implemented and validated through different levels of testing.

The testing activities include unit tests for individual modules, integration tests to verify interactions between components, and functional tests to validate the overall system behavior. Each test case is linked to its corresponding requirement, ensuring full traceability and coverage.

The results presented in this document provide evidence of the system's correctness, robustness, and compliance with the specified requirements.

2 Test Case Repository Overview

The complete set of test cases is maintained in an external spreadsheet, available in the Appendix (see Section 7). This spreadsheet contains the full definition of all test artifacts, including detailed descriptions, identifiers, and requirement traceability.

The spreadsheet is organized into the following tabs:

- **MASTER REQUIREMENTS**: This tab provides the mapping of all Software Low-Level (SW-LL) requirements. It serves as the baseline reference for traceability and ensures that all requirements are properly identified and covered by the test cases.

- **Unit Test Tabs (UT modules)**: Each module has its own unit test tab (e.g., Operation, Fuel, Timer, CO2, Safety Inhibit, Enable/Disable). These tabs contain:

  - Unique test case IDs
  - Test descriptions
  - Expected behavior
  - Traceability to SW Low-Level requirements

  The objective of these tests is to validate each module in isolation.

- **Integration Tests**: This tab contains integration test cases that validate the interaction between multiple modules. Each test case includes:

  - Test scenario description
  - Expected system behavior
  - Traceability to System Low-Level (SYS-LL) requirements

- **Functional Tests**: This tab contains high-level functional test scenarios that validate the system behavior from an end-to-end perspective. These test cases include:
  - Functional scenarios
  - Expected outputs
  - Traceability to System High-Level (SYS-HL) and System Low-Level (SYS-LL) requirements

3 Automation Testing Tools

The verification strategy relies on three complementary automated tools, each targeting a different class of defect. Static analysis enforces coding-standard compliance at the source level, structural coverage quantifies how thoroughly the test suite exercises the code, and mutation testing evaluates the fault-detection strength of the existing assertions. All three tools run inside the same Docker-based environment described in Section 4 and are orchestrated through a single `Makefile`.

3.1 Misra C - Static Analysis

MISRA C:2012 is a widely adopted coding standard for safety-critical embedded software, originally developed for the automotive industry and now referenced by functional-safety standards such as ISO 26262 and IEC 61508. The project enforces it through **Cppcheck** with the built-in `misra` add-on, invoked by the Makefile target `make misra`.

```
cppcheck \
    --std=c99 \
    --addon=misra \
    --error-exitcode=1 \
    --enable=warning,style,performance,portability \
    --suppress=missingIncludeSystem \
    -I include/ \
    --xml --xml-version=2 \
    src/*.c 2>build/cppcheck-report.xml
```

Key aspects of the configuration:

- **Standard:** C99 (`--std=c99`), matching the compiler setting used throughout the project.
- **Checks enabled:** warnings, style, performance and portability, in addition to the MISRA rules.

- **Report format:** XML v2, stored in `build/cppcheck-report.xml`. In CI, this artifact is uploaded and a human-readable summary is written to the GitHub Actions job summary.
- **Gate behaviour:** the flag `--error-exitcode=1` causes the build to fail whenever a violation is detected, preventing non-compliant code from being merged.

3.1.1 Supported MISRA C:2012 Rules

The Cppcheck MISRA add-on is implemented as a Python script (`misra.py` [1]) that analyses Cppcheck's internal AST dump and enforces approximately 143 of the 175 MISRA C:2012 rules (including Amendments 1 and 2). Each rule is implemented as an individual checker function (e.g. `misra_5_1`, `misra_12_3`, `misra_21_3`), making it straightforward to trace a reported violation back to the corresponding standard clause.

The rules are classified by the MISRA standard into three obligation levels:

- **Mandatory** – must always be followed; no deviation is permitted (e.g. Rule 1.3, Rule 17.3).
- **Required** – must be followed unless a formal deviation is documented and justified (e.g. Rule 8.4, Rule 10.4, Rule 21.3).
- **Advisory** – recommended best practice; formal deviation is not required but compliance is encouraged (e.g. Rule 2.5, Rule 15.7).

The checked rules span all major categories of the standard, including but not limited to:

- **Environment and compilation** (Dir 1.1, Rule 1.2, Rule 1.4) – undefined and implementation-defined behaviour, language extensions.
- **Unused code** (Rules 2.2–2.7) – dead code, unused variables, parameters, tags and macros.
- **Comments** (Rule 3.1) – nested comments and comment sequences.
- **Character sets and literals** (Rules 4.1–4.2, 7.1–7.4) – escape sequences, trigraphs, octal constants, string literal mutability.
- **Identifiers** (Rules 5.1–5.9) – uniqueness, scope visibility, tag and typedef naming.
- **Types** (Rules 6.1–6.2, 10.x) – bit-fields, implicit conversions, essential type model, casts.

[1] `https://github.com/danmar/cppcheck/blob/main/addons/misra.py`

- **Declarations and definitions** (Rules 8.x) – external linkage, tentative definitions, compatible declarations, static/inline usage.
- **Initialisation** (Rules 9.x) – partially and incorrectly initialised aggregates.
- **Pointers** (Rules 11.x, 18.x) – pointer conversions, arithmetic, `const` qualifiers.
- **Expressions and control flow** (Rules 12.x–15.x) – evaluation order, side-effects, implicit boolean conversions, switch/if structure, `goto` restrictions.
- **Functions** (Rules 17.x) – prototypes, return statements, stdarg prohibition.
- **Preprocessing** (Rules 20.x) – macro hygiene, `#include` guards, `#define`/`#undef` restrictions.
- **Standard libraries** (Rules 21.x–22.x) – banned headers (`<stdio.h>`, `<stdlib.h>`, `<signal.h>`), dynamic memory, `errno` usage, resource management.

Because the MISRA standard is proprietary, Cppcheck only reports the rule number of each violation (e.g. `[c2012-21.3]`) without reproducing the rule text. Full descriptions can optionally be provided via the `--rule-texts` flag pointing to a local copy of the standard [2].

For a detailed reference of all rules in the MISRA C:2012 standard, consult the official publication [3].

3.2 `llvm-cov` - Statement, Branch and MC/DC Coverage

Structural coverage is measured with the LLVM toolchain (version 18), using `llvm-profdata` to merge raw profiles and `llvm-cov` to produce reports. Three coverage criteria are collected simultaneously for every module under test:

1. **Statement (line) coverage** – percentage of executable lines reached.
2. **Branch coverage** – percentage of Boolean branch outcomes (true/false) exercised.
3. **MC/DC (Modified Condition / Decision Coverage)** – verifies that each condition within a decision independently affects the decision outcome, as required by standards such as DO-178C Level A and ISO 26262 ASIL D.

The instrumentation is enabled at compile time through three Clang flags:

[2] See the add-on's documentation: `https://github.com/danmar/cppcheck/blob/main/addons/README.md`

[3] `https://www.misra.org.uk/app/uploads/2023/03/MISRA-C-2012-AMD4.pdf`

```
COV_FLAGS := -fprofile-instr-generate \
             -fcoverage-mapping       \
             -fcoverage-mcdc          \
             -g -O0
```

- `-fprofile-instr-generate`: instruments the binary to emit a `.profraw` file at run time.
- `-fcoverage-mapping`: embeds source-level mapping data so that `llvm-cov` can attribute counters to lines and branches.
- `-fcoverage-mcdc`: enables the MC/DC-specific instrumentation introduced in LLVM 18, which tracks the truth table of each decision.

The `make coverage` target automates the full workflow per module:

1. Compile each test binary with `COV_FLAGS`.
2. Execute the test and produce a `.profraw` file.
3. Merge the profile with `llvm-profdata merge -sparse`.
4. Generate a per-module HTML report via `llvm-cov show ... -show-branches=count -show-mcdc -format=html`.
5. Export a JSON summary with `llvm-cov export -summary-only` and extract the percentage for each metric (lines, branches, mcdc).
6. Compare every metric against the configurable threshold `COV_THRESHOLD` (default 80 %). If any module falls below the threshold the build fails.
7. Produce a combined HTML dashboard (`build/coverage/index.html`) summarising all modules with colour-coded pass/fail indicators.

3.3 mull - Mutation Testing

The production sources are compiled with additional flags that enable Mull's instrumentation pass:

```
MUT_FLAGS := -g -O0 -fno-omit-frame-pointer \
  -fpass-plugin=/usr/lib/mull-ir-frontend-18 \
  -grecord-command-line
```

- `-fpass-plugin`: loads Mull's LLVM pass at compile time, embedding mutation metadata directly into the object files.

- `-grecord-command-line`: records the original compiler invocation so that `mull-runner` can replay it during mutation.
- `-fno-omit-frame-pointer`: preserves frame pointers for reliable stack traces when a mutant causes a crash.

Each production source file is compiled into a separate object file (`.o`) under `build/mutation/`, so Mull can replay individual compilation commands independently.

3.3.1 Mutator configuration

The project-wide file `mull.yml` defines the active mutators:

```
mutators:
  - cxx_add_to_sub        # a + b  ->  a - b
  - cxx_sub_to_add        # a - b  ->  a + b
  - cxx_mul_to_div        # a * b  ->  a / b
  - cxx_div_to_mul        # a / b  ->  a * b
  - cxx_eq_to_ne          # a == b ->  a != b
  - cxx_ne_to_eq          # a != b ->  a == b
  - cxx_gt_to_ge          # a > b  ->  a >= b
  - cxx_gt_to_le          # a > b  ->  a <= b
  - cxx_ge_to_gt          # a >= b ->  a > b
  - cxx_ge_to_lt          # a >= b ->  a < b
  - cxx_lt_to_ge          # a < b  ->  a >= b
  - cxx_lt_to_le          # a < b  ->  a <= b
  - cxx_le_to_gt          # a <= b ->  a > b
  - cxx_le_to_lt          # a <= b ->  a < b
  - cxx_post_inc_to_post_dec  # i++ -> i--
  - cxx_post_dec_to_post_inc  # i-- -> i++
  - cxx_minus_to_noop     # -x  ->  x
```

The 17 mutators cover arithmetic replacements, relational-operator boundary shifts, increment/decrement inversions, and unary-minus removal. Paths excluded from mutation include the Unity test framework (`tests/vendor/.*`), the test harnesses themselves (`tests/test_.*`), and the Qt/QML cluster code (`src/Cluster_SIL/.*`).

3.3.2 Execution flow

The `make mutation` target iterates over each test binary and:

1. Creates a per-module working directory (`build/mutation/run_<module>`).

2. Generates a tailored `mull.yml` that restricts the `includePaths` to the single source file under test (e.g. `.*/src/ss_timer\.c`).
3. Invokes `mull-runner-18` with `--mutation-score-threshold=80` (configurable via `MUT_THRESHOLD`).
4. If any module's mutation score falls below the threshold, the overall target exits with a non-zero status.

4 Environment Configuration - Make and CI

All build, analysis and test steps are containerised using a Docker image defined in `.devcontainer/Dockerfile`, based on Ubuntu 24.04. The image bundles every tool required by the pipeline:

- **Compiler:** Clang 18 with LLVM 18 (`clang-18`, `llvm-cov-18`, `llvm-profdata-18`).
- **Static analysis:** Cppcheck (with MISRA add-on).
- **Formatting:** `clang-format`.
- **Mutation testing:** Mull 0.33.0 (`mull-runner-18`, `mull-ir-frontend-18`).
- **Utilities:** GNU Make, CMake, `jq`, Python 3.

Using a single container image guarantees reproducibility: the same toolchain versions run on every developer workstation and in CI.

4.1 Make's rules

The project's `Makefile` exposes the following top-level targets:

Table 1: Makefile targets and their purposes.

Target	Description
`build`	Compile all sources and link the application binary (`build/start_stop_demo`).
`misra`	Run Cppcheck with the MISRA C:2012 add-on and produce an XML report.
`format-check`	Verify source formatting against `.clang-format` rules (no modifications).
`test`	Build and execute all Unity unit-test binaries; fail on first error.
`coverage`	Instrument, run and report statement, branch and MC/DC coverage.
`mutation`	Compile with the Mull IR-frontend plugin and run mutation analysis.
`integration`	Build and run the system-level integration test.
`clean`	Remove all build artefacts (`build/`).

The compiler is set to `clang-18` with strict C99 diagnostics (`-std=c99 -Wall -Wextra -Wpedantic -Werror`). The Unity test-framework object (`unity.o`) is compiled *without* `-Werror` to tolerate upstream warnings without patching the third-party source.

Separate build directories are used for each concern (`build/` for production, `build/coverage/` for instrumented binaries, `build/mutation/` for Mull objects), keeping artefacts isolated and making `make clean` trivial.

4.2 CI´s jobs

Continuous Integration is implemented with **GitHub Actions** and split across two workflow files, each triggered on pushes and pull requests to `main`, as well as manual dispatch.

4.2.1 Workflow 1: `ci-static-analysis.yml`

This workflow focuses on build correctness and code quality. It begins with a lightweight `check-files` job that verifies whether the required source files exist, then conditionally runs three parallel jobs:

1. **Build** (`build`) – Builds the Docker image, compiles the project with `make build`, and executes the resulting binary to confirm a clean build. Only runs when `src/main.c` is present.

2. **Format Check** (`format-check`) – Runs `make format-check` inside the container to ensure all source and header files conform to the `.clang-format` style.

3. **MISRA Analysis** (`misra-analysis`) – Executes `make misra`, uploads the XML report as a build artefact, and writes a human-readable violation summary to the GitHub Actions job summary. The job fails if any MISRA violation is detected.

4.2.2 Workflow 2: `ci-tests.yml`

This workflow validates functional correctness and test quality:

1. **Unit Tests** (`unit-tests`) – Checks out the repository with submodules (Unity framework), builds the Docker image, and runs `make test`.

2. **Coverage** (`coverage`) – Depends on `unit-tests`. Runs `make coverage` and uploads the full `build/coverage/` directory (excluding raw profile data) as a downloadable artefact.

3. **Mutation Testing** (`mutation`) – Also depends on `unit-tests`. Runs `make mutation` to verify that the test suite achieves the required mutation score.

5 Testings Cases

5.1 Unity Tests

For each software module, a dedicated set of unit tests was developed to validate its behavior in isolation. These tests were designed to ensure that all functional requirements are correctly implemented, covering nominal scenarios, boundary conditions, and robustness cases. The unit tests also aim to achieve high levels of statement, branch, and MC/DC coverage, providing confidence in the correctness and reliability of each module independently.

5.1.1 Operation Module

The Operation module is responsible for managing the core state machine of the Start/Stop system. It evaluates system inputs and operational conditions to control state transitions and generate engine stop and restart requests.

The table below presents the unit test cases developed for the Operation module. These tests validate the correct behavior of the state machine, including transitions between states, evaluation of stop and restart conditions, and robustness against invalid or boundary inputs.

Table 2: Operational Module Tests

Unit Test ID	Unit Test Name	Test Objective	Test Status
UT-OP-01	test_SS_Operation_requests_engine_stop_when_stop_conditions_are_met	Verify that the module generates an engine stop request when all autostop conditions are satisfied	PASS
UT-OP-02	test_SS_Operation_requests_restart_when_accelerator_pressed	Verify restart request due to accelerator input during AutoStop	PASS
UT-OP-03	test_SS_Operation_requests_restart_when_brake_released	Verify restart request when brake is released during AutoStop	PASS
UT-OP-04	test_SS_Operation_requests_restart_when_gear_changes	Verify restart request when gear changes away from Drive during AutoStop	PASS
UT-OP-05	test_SS_Operation_requests_restart_when_timeout_is_reached	Verify restart request when autostop timeout is reached	PASS
UT-OP-06	test_SS_Operation_requests_restart_when_ss_is_disabled	Verify restart request when Start-Stop is disabled during AutoStop	PASS
UT-OP-07	test_SS_Operation_Init_sets_vehicle_off	Verify initial state after initialization	PASS
UT-OP-08	test_SS_Operation_goes_to_engine_running_when_ignition_turns_on	Verify transition from VEHICLE_OFF to ENGINE_RUNNING	PASS
UT-OP-09	test_SS_Operation_returns_to_vehicle_off_when_ignition_turns_off	Verify transition back to VEHICLE_OFF when ignition turns off	PASS
UT-OP-10	test_SS_Operation_blocks_restart_when_safe_stop_is_active	Verify that restart is blocked when safe_stop = TRUE	PASS
UT-OP-11	test_SS_Operation_does_not_request_stop_when_conditions_are_not_met	Verify no stop request is generated when one stop condition is invalid	PASS
UT-OP-12	test_SS_Operation_handles_null_pointers	Verify robustness for null pointer inputs	PASS
UT-OP-13	test_SS_Operation_does_not_stop_when_ignition_is_off	Falha da condição de ignição	PASS
UT-OP-14	test_SS_Operation_does_not_stop_when_ss_is_disabled_before_autostop	Falha da condição de enable	PASS
UT-OP-15	test_SS_Operation_does_not_stop_when_autostop_is_not_allowed	Falha da condição autostop_allowed	PASS
UT-OP-16	test_SS_Operation_does_not_stop_when_gear_is_not_drive	Falha da condição de marcha	PASS
UT-OP-17	test_SS_Operation_does_not_stop_when_speed_is_above_limit	Falha da condição de velocidade	PASS
UT-OP-18	test_SS_Operation_remains_in_autostop_when_no_restart_condition_is_met	Caminho negativo de restart	PASS
UT-OP-19	test_SS_Operation_remains_in_autostop_when_restart_condition_is_met_but_safe_stop_is_true	Restart bloqueado por safe stop	PASS
UT-OP-20	test_SS_Operation_goes_to_vehicle_off_when_ignition_turns_off_during_autostop	Caminho de ignição off em autostop	PASS
UT-OP-21	test_SS_Operation_GetState_returns_current_state	Cobrir getter	PASS
UT-OP-22	test_SS_Operation_IsAutoStopActiveState_returns_true_when_in_autostop	Cobrir getter	PASS
UT-OP-23	test_SS_Operation_IsAutoStopActiveState_returns_false_when_not_in_autostop	Cobrir getter	PASS
UT-OP-24	test_SS_Operation_default_case_recovers_to_vehicle_off	Verify recovery to VEHICLE_OFF when internal state is invalid (switch default branch)	PASS
UT-OP-25	test_SS_Operation_helper_stop_enabled_returns_false_when_input_is_null	Verify helper predicate SS_StopEnabled returns FALSE when input pointer is NULL	PASS
UT-OP-26	test_SS_Operation_helper_stop_allowed_returns_false_when_input_is_null	Verify helper predicate SS_StopAllowed returns FALSE when input pointer is NULL	PASS
UT-OP-27	test_SS_Operation_helper_stop_gear_in_drive_returns_false_when_input_is_null	Verify helper predicate SS_StopGearInDrive returns FALSE when input pointer is NULL	PASS
UT-OP-28	test_SS_Operation_helper_stop_brake_pressed_returns_false_when_input_is_null	Verify helper predicate SS_StopBrakePressed returns FALSE when input pointer is NULL	PASS
UT-OP-29	test_SS_Operation_helper_stop_speed_within_limit_returns_false_when_input_is_null	Verify helper predicate SS_StopSpeedWithinLimit returns FALSE when input pointer is NULL	PASS
UT-OP-30	test_SS_Operation_helper_restart_by_accelerator_returns_false_when_input_is_null	Verify helper predicate SS_RestartByAccelerator returns FALSE when input pointer is NULL	PASS
UT-OP-31	test_SS_Operation_helper_restart_by_brake_release_returns_false_when_input_is_null	Verify helper predicate SS_RestartByBrakeRelease returns FALSE when input pointer is NULL	PASS
UT-OP-32	test_SS_Operation_helper_restart_by_gear_change_returns_false_when_input_is_null	Verify helper predicate SS_RestartByGearChange returns FALSE when input pointer is NULL	PASS
UT-OP-33	test_SS_Operation_helper_restart_by_disable_returns_false_when_input_is_null	Verify helper predicate SS_RestartByDisable returns FALSE when input pointer is NULL	PASS
UT-OP-34	test_SS_Operation_helper_restart_by_timeout_returns_false_when_input_is_null	Verify helper predicate SS_RestartByTimeout returns FALSE when input pointer is NULL	PASS
UT-OP-35	test_SS_Operation_stop_condition_false_when_ignition_is_off	Verify MC/DC for stop condition: ignition_on independently forces the stop condition result to FALSE	PASS
UT-OP-36	test_SS_Operation_helper_stop_ignition_on_returns_false_when_input_is_null	Verify robustness and MC/DC: StopIgnitionOn returns FALSE when input is NULL, covering condition (in != NULL)	PASS

5.1.2 Inhibit Module

The Inhibit Module evaluates safety and operational conditions to determine whether the Start/Stop function is allowed. The function IhibitSafetyConditions was unit tested across a wide range of scenarios to ensure correctness and compliance with low-level requirements.

The table below presents the unit test cases developed for the Inhibit module. These tests validate the evaluation of key safety inputs (inclination, door, seatbelt, gear, and system status), including nominal operation, fault conditions, boundary values, and extreme cases. The expected behavior ensures that AutoStop is permitted only when all safety conditions are simultaneously satisfied.

Table 3: Inhibit Module Tests

Unit Test ID	Unit Test Name	Test Objective	Test Status
UT-ISC-01	test_inhibit_safety_conditions_all_conditions_true	Verify system enables Start/Stop when all inputs are valid.	PASS
UT-ISC-02	test_inhibit_safety_conditions_high_inclination	Verify inhibition due to high inclination.	PASS
UT-ISC-03	test_inhibit_safety_conditions_door_open	Verify inhibition when the door status is FALSE (open).	PASS
UT-ISC-04	test_inhibit_safety_conditions_seatbelt_unfastened	Verify inhibition when seatbelt status is FALSE.	PASS
UT-ISC-05	test_inhibit_safety_conditions_wrong_gear	Verify inhibition when gear is not SS_GEAR_D.	PASS
UT-ISC-06	test_inhibit_safety_conditions_disabled_system	Verify inhibition when system_enabled input is FALSE.	PASS
UT-ISC-07	test_inhibit_safety_conditions_boundary_inclination	Verify boundary condition for inclination limit.	PASS
UT-ISC-08/09	test_inhibit_safety_conditions_zero_inclination	Verify operation on flat and downhill surfaces.	PASS
UT-ISC-16	test_inhibit_safety_conditions_inclination_just_above_limit	Verify threshold sensitivity for inclination.	PASS
UT-ISC-34	test_standstill_management_set_condition	Verify the detection of vehicle stationary state.	PASS
UT-ISC-36	test_standstill_management_reset_condition	Verify Standstill state transition to FALSE.	PASS
UT-ISC-39	test_drivecycle_memory_block_high_speed	Verify that the cycle memory is set after exceeding speed limits.	PASS
UT-ISC-41	test_drivecycle_memory_block_reset_memory	Verify memory reset on ignition cycle.	PASS
UT-STM-01	test_standstill_management_initial_false	Verify initial standstill state is FALSE.	PASS
UT-STM-02	test_standstill_management_set_condition	Verify state transition to TRUE after meeting conditions.	PASS
UT-STM-03	test_standstill_management_reset_condition	Verify state transition to FALSE when motion is detected.	PASS
UT-STM-04	test_standstill_management_reset_after_set	Verify immediate reset capability.	PASS
UT-STM-05	test_standstill_management_hold_state	Verify state latching/holding behavior.	PASS
UT-DMB-01	test_drivecycle_memory_block_ignition_off	Verify memory is FALSE when ignition is OFF.	PASS
UT-DMB-02	test_drivecycle_memory_block_engine_stop_request	Verify memory behavior during stop requests.	PASS
UT-DMB-03	test_drivecycle_memory_block_high_speed	Verify flag is set upon reaching target speed.	PASS
UT-DMB-04	test_drivecycle_memory_block_memory_effect	Verify latching of speed threshold flag.	PASS
UT-DMB-05	test_drivecycle_memory_block_reset_memory	Verify full memory reset logic.	PASS
UT-SRF-01	test_sr_flip_flop_set	Verify Q becomes TRUE when S is TRUE.	PASS
UT-SRF-02	test_sr_flip_flop_reset	Verify Q becomes FALSE when R is TRUE.	PASS
UT-SRF-03	test_sr_flip_flop_hold_s0_r0	Verify state retention with no inputs.	PASS
UT-SRF-05	test_sr_flip_flop_null_state	Verify robust failure (FALSE) with NULL state.	PASS

5.1.3 CO2 Module

The CO2 Module estimates the amount of carbon dioxide emissions avoided during each AutoStop event. It tracks the accumulated stop time, computes CO2 savings using the WLTP idle factor, and controls the cluster display flags accordingly. The table below presents the unit test cases developed for the CO2 module. These tests validate time accumulation during active AutoStop periods, CO2 calculation accuracy, display and hide flag behavior, initialization state, ignition-off reset logic, negative cases, and boundary/stress scenarios such as sustained operation over 1000 consecutive cycles.

Table 4: CO2 Module Tests

Unit Test ID	Unit Test Name	Test Objective	Test Status
UT-CO2-01	test_co2_autostop_active_increments_time_by_task_period	Verify accumulated AutoStop time is incremented by one task period (10ms) when AutoStop is active.	PASS
UT-CO2-02	test_co2_autostop_active_increments_time_multiple_calls	Verify accumulated AutoStop time over multiple consecutive active cycles.	PASS
UT-CO2-03	test_co2_ignition_off_resets_time	Verify that accumulated AutoStop time resets to zero when ignition is turned OFF.	PASS
UT-CO2-04	test_co2_autostop_active_calculates_avoided_grams	Verify CO2 avoided calculation based on accumulated AutoStop time and WLTP idle factor.	PASS
UT-CO2-05	test_co2_autostop_active_enables_display	Verify display flag is enabled when AutoStop is active.	PASS
UT-CO2-06	test_co2_autostop_inactive_enables_hide	Verify hide flag is enabled when AutoStop is inactive.	PASS
UT-CO2-07	test_co2_autostop_inactive_disables_display	Verify display flag is disabled when AutoStop is inactive.	PASS
UT-CO2-08	test_co2_ignition_off_resets_avoided_grams	Verify CO2 avoided value resets to zero when ignition is OFF.	PASS
UT-CO2-09	test_co2_ignition_off_disables_display	Verify display flag is disabled when ignition is OFF.	PASS
UT-CO2-10	test_co2_ignition_off_enables_hide	Verify hide flag is enabled when ignition is OFF.	PASS
UT-CO2-11	test_co2_init_resets_accumulated_time	Verify initialization resets accumulated AutoStop time to zero.	PASS
UT-CO2-12	test_co2_init_resets_avoided_grams	Verify initialization resets CO2 avoided grams to zero.	PASS
UT-CO2-13	test_co2_init_display_disabled	Verify initialization disables display flag.	PASS
UT-CO2-14	test_co2_init_hide_enabled	Verify initialization enables hide flag.	PASS
UT-CO2-15	test_co2_autostop_inactive_does_not_increment_time	Verify accumulated time does not increase when AutoStop is inactive.	PASS
UT-CO2-16	test_co2_avoided_grams_zero_when_autostop_inactive	Verify CO2 avoided remains zero while AutoStop is inactive.	PASS
UT-CO2-17	test_co2_autostop_active_disables_hide	Verify hide flag is disabled when AutoStop is active.	PASS
UT-CO2-18	test_co2_transition_active_inactive_active_preserves_time	Verify accumulated time is preserved across active → inactive → active transitions.	PASS
UT-CO2-19	test_co2_ignition_cycle_resets_and_restarts_clean	Verify full ignition cycle resets all state and module restarts from zero.	PASS
UT-CO2-20	test_co2_avoided_grams_accumulates_over_multiple_cycles	Verify CO2 avoided calculation accuracy after multiple consecutive active cycles.	PASS
UT-CO2-21	test_co2_ignition_off_with_autostop_true_still_resets	Verify ignition OFF resets all state even when autostop_active is TRUE (equivalence partition).	PASS
UT-CO2-22	test_co2_boundary_long_accumulation_1000_cycles	Verify accumulated time and CO2 calculation remain accurate after 1000 consecutive active cycles (boundary/stress).	PASS

5.1.4 Fuel Module

The Fuel module is responsible for estimating and tracking the amount of fuel saved during AutoStop events. It computes fuel savings based on accumulated AutoStop time and calibrated consumption rates, as well as controlling the visibility of this information.

The table below presents the unit test cases developed for the Fuel module. These tests verify the correct accumulation of AutoStop time, accurate computation of fuel savings, proper reset behavior, and correct handling of display enable and disable conditions.

Table 5: Fuel Tests

Unit Test ID	Unit Test Name	Test Objective	Test Status
UT-FUEL-01	test_fuel_autostop_active_increments_time_by_task_period	Verify that AutoStop accumulated time is incremented by one task period when AutoStop is active	PASS
UT-FUEL-02	test_fuel_autostop_active_increments_time_multiple_calls	Verify accumulated AutoStop time over multiple active cycles	PASS
UT-FUEL-03	test_fuel_ignition_off_resets_time	Verify that accumulated AutoStop time resets when ignition is OFF	PASS
UT-FUEL-04	test_fuel_autostop_active_calculates_saved_liters	Verify fuel saved calculation based on accumulated AutoStop time and idle fuel rate	PASS
UT-FUEL-05	test_fuel_autostop_active_enables_display	Verify display flag is enabled when AutoStop is active	PASS
UT-FUEL-06	test_fuel_autostop_inactive_enables_hide	Verify hide flag is enabled when AutoStop is inactive	PASS
UT-FUEL-07	test_fuel_ignition_off_resets_saved_liters	Verify fuel saved value resets when ignition is OFF	PASS
UT-FUEL-08	test_fuel_init_resets_accumulated_time	Verify initialization resets accumulated AutoStop time	PASS
UT-FUEL-09	test_fuel_init_resets_saved_liters	Verify initialization resets fuel saved value	PASS
UT-FUEL-10	test_fuel_init_display_disabled	Verify initialization disables display flag	PASS
UT-FUEL-11	test_fuel_init_hide_enabled	Verify initialization enables hide flag	PASS
UT-FUEL-12	test_fuel_autostop_inactive_disables_display	Verify display flag is disabled when AutoStop is inactive	PASS
UT-FUEL-13	test_fuel_ignition_off_disables_display	Verify display flag is disabled when ignition is OFF	PASS
UT-FUEL-14	test_fuel_ignition_off_enables_hide	Verify hide flag is enabled when ignition is OFF	PASS
UT-FUEL-15	test_fuel_autostop_inactive_does_not_increment_time	Verify accumulated time does not increase when AutoStop is inactive	PASS
UT-FUEL-16	test_fuel_saved_liters_zero_when_autostop_inactive	Verify fuel saved remains zero while AutoStop is inactive	PASS
UT-FUEL-17	test_fuel_autostop_active_disables_hide	Verify hide flag is disabled when AutoStop is active	PASS

5.1.5 Enable/Disable Module

The Enable/Disable Module is responsible for managing the user interface and the core activation logic of the Start-Stop system via a physical button and ignition status. A comprehensive set of unit tests was developed to validate the Finite State Machine (FSM), the debounce mechanism, and safety interlocks.

The table below presents the unit test cases developed for the Enable/Disable module. These tests verify the correct behavior of the debounce Finite State Machine (FSM), the accurate toggling of the system state via button input, the mandatory reset to the enabled state upon ignition rising edges, and the proper synchronization of the HMI LED feedback.

Table 6: Enable-Disable Module Tests

Unit Test ID	Unit Test Name	Test Objective	Test Status
UT-ED-01	test_init_ss_enabled_default	Verify that the Start-Stop system is enabled by default upon power-on reset.	PASS
UT-ED-02	test_init_fsm_idle_state	Confirm that the internal FSM starts in the ST_IDLE state during initialization.	PASS
UT-ED-03	test_init_debounce_counter_reset	Verify that the debounce counter is initialized to zero to prevent false triggers.	PASS
UT-ED-04	test_button_toggle_logic_inverts_state	Validate that a valid button press successfully inverts the system enable/disable state.	PASS
UT-ED-05	test_button_toggle_updates_hmi_led	Ensure that the HMI LED output is correctly updated to reflect the current system state.	PASS
UT-ED-06	test_button_noise_rejection_30ms	Confirm that a button signal held for 30ms (shorter than threshold) is rejected as noise.	PASS
UT-ED-07	test_button_noise_rejection_fsm_behavior	Verify that the FSM remains in debouncing state during short pulses without triggering logic.	PASS
UT-ED-08	test_button_release_return_to_idle	Ensure the FSM returns to IDLE state if a noise spike is released before the debounce time.	PASS
UT-ED-09	test_permissions_disabled_when_ss_off	Verify that Autostop and Autostart permissions are revoked when the system state is set to FALSE.	PASS
UT-ED-10	test_permissions_disabled_without_ignition	Confirm that the system output remains FALSE even if enabled if the ignition signal is absent.	PASS
UT-ED-11	test_ignition_rising_edge_forces_enable	Validate that any rising edge on ignition forces the system back to the Enabled state for safety.	PASS
UT-ED-12	test_led_update_cmd_pulsed_output	Verify that the LED_Update_Cmd is triggered only during state transitions.	PASS

5.1.6 Timer Module

The Timer Module is responsible for tracking the duration of Autostop events to prevent excessive engine-off periods and ensure battery health. A robust set of unit tests was implemented to validate time accumulation, timeout detection, and the reset mechanism.

The table below presents the unit test cases developed for the Timer module. These tests verify the precise accumulation of AutoStop time based on the task period, the correct detection of the timeout threshold, the immediate reset of the timer and flags when Autostop is inactive, and the robustness of the saturation logic to prevent variable overflow.

Table 7: Timer Module Tests

Unit Test ID	Unit Test Name	Test Objective	Test Status
UT-TM-01	test_timer_timeout_reached_at_threshold	Verify that the timeout flag is set to TRUE exactly when the accumulated time reaches 180,000ms.	PASS
UT-TM-02	test_timer_increment_by_task_period	Validate that each call to the timer function increments the accumulated time by the defined task period (10ms/100ms).	PASS
UT-TM-03	test_timer_reset_on_ignition_off	Confirm that the accumulated time and timeout flag are fully reset when the ignition signal is lost.	PASS
UT-TM-04	test_timer_reset_on_autostop_inactive	Ensure the timer resets to zero whenever the Autostop becomes inactive, preventing cumulative errors between cycles.	PASS
UT-TM-05	test_timer_init_state	Verify that the internal global variables are correctly zeroed out during the module's power-on initialization.	PASS
UT-TM-06	test_timer_saturation_at_max_limit	Ensure the timer saturates at the maximum calibrated value and does not overflow, maintaining a safe and stable output.	PASS

5.2 Integration Test Cases

Integration tests were developed to verify the correct interaction between the different software modules of the Start-Stop System. These tests focus on validating the system behavior when multiple components operate together, ensuring that data flow and control logic across modules are correctly implemented.

The integration test cases cover key system functionalities such as AutoStop activation, engine restart conditions, safety inhibitions, and the interaction with estimation modules (CO_2 and Fuel). The objective is to ensure that the combined behavior of the modules satisfies the specified System Low-Level requirements.

The table below presents the integration test cases, including their identifiers, names, modules involved, and objectives to the corresponding system requirements.

Table 8: Integration Tests

Test ID	Test Name	Test Objective	Test Status
IT-SYS-LL-01	test_integration_autostop_enables_estimation_modules	Verify that inhibit logic generates AutoStopAllowed, Operation enters AutoStop, Timer increments, and CO2/Fuel estimations are updated while AutoStop is active	PASS
IT-SYS-LL-02	test_integration_restart_disables_autostop_estimations	Verify that a restart trigger during AutoStop generates EngineRestartRequest and disables CO2/Fuel display logic	PASS
IT-SYS-LL-03	test_integration_ignition_off_resets_estimation_outputs	Verify that ignition OFF transitions the system to VEHICLE_OFF and resets accumulated AutoStop time, CO2 avoided, and fuel saved values	PASS
IT-SYS-LL-04	test_integration_safe_stop_blocks_restart	Verify that SafeStop generated by inhibit/safety logic is consumed by Operation and prevents restart even when a restart trigger is present	PASS
IT-SYS-LL-05	test_integration_drive_cycle_memory_controls_autostop_permission	Verify that AutoStop is blocked before the drive cycle memory condition and allowed after the vehicle has exceeded the required speed threshold	PASS
IT-SYS-LL-06	test_integration_ss_enable_toggle_affects_operation_behavior	Verify that button toggle updates SS_Enable and that Operation behavior changes accordingly when Start/Stop is disabled	PASS
IT-SYS-LL-07	test_integration_inhibit_conditions_block_autostop	Verify that inhibit conditions such as invalid inclination, door, seatbelt, or gear prevent AutoStopAllowed and block engine stop request	PASS

5.3 Functional Test Cases

The primary objective of this phase is to evaluate the system's overall behavior, ensuring that the design fully complies with the requirements under simulated real-world conditions.

To ensure a the validation step was optimized, a systematic testing routine has been structured, which will be executed via the *dashboard* (Human-Machine Interface) previously developed within the Simulink environment. This interface will serve as the central tool for dynamic input injection into the system, emulating sensor signals and driver interactions, enabling the direct observation of the control loop's responses and attesting to the reliability and robustness of the final application.

Due to limitations imposed by the development environment (WSL Container) utilized on this project, the communication between the data input, the start/stop system and cluster application was abstracted using User Datagram Protocol (UDP) instead Controller Area Network (CAN).

During the execution of the test routines, monitoring is conducted in a hybrid manner: through the Command Window, where execution logs and internal model signals are verified, and simultaneously via the cluster app. This integration makes it possible to instantly check state transitions and the behavior of critical variables, such as the accumulated AutoStop time and fuel estimations, as illustrated in Figure 1.

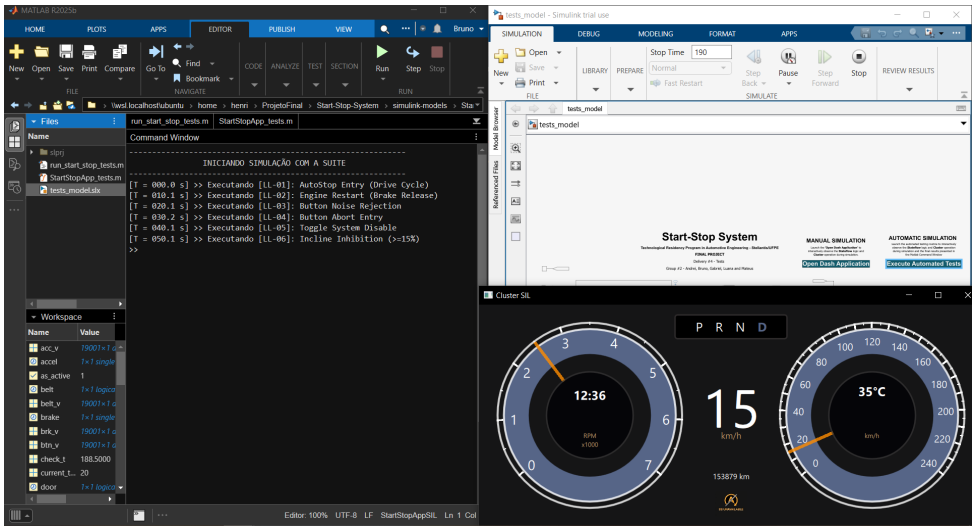

Figure 1: Simulink Testing Tool

Table 9: Funtional Tests

Test ID	Test Name	Test Objective	Test Status
FT-SYS-LL-01	test_engine_stop_request_when_autostop_conditions_are_met	Verify that processed inputs and inhibit logic interact correctly with the Operation Module to generate EngineStopRequest when all AutoStop conditions are satisfied	PASS
FT-SYS-LL-02	test_engine_restart_request_when_restart_condition_is_met	Verify that restart trigger conditions propagate correctly through integrated modules and generate EngineRestartRequest during AutoStop	PASS
FT-SYS-LL-03	test_toggle_ss_enable_when_button_rising_edge_is_detected	Verify that debounce and control logic correctly toggle the Start/Stop enable state	PASS
FT-SYS-LL-04	test_state_transition_propagation_between_control_and_operation	Verify that Start/Stop enable state transitions correctly affect the Operation Module behavior	PASS
FT-SYS-LL-05	test_activation_status_reflects_ss_enable_state	Verify that the Start/Stop enable/disable state is correctly maintained and propagated by control logic	PASS
FT-SYS-LL-06	test_unavailable_condition_blocks_operation	Verify that inhibit conditions correctly prevent Start/Stop operation when the system is unavailable	PASS
FT-SYS-LL-07	test_co2_computation_enabled_when_autostop_is_active	Verify that AutoStop state enables CO2 accumulation and computation across modules	PASS
FT-SYS-LL-08	test_co2_computation_disabled_when_autostop_is_inactive	Verify that CO2 accumulation/display logic is disabled when AutoStop becomes inactive	PASS
FT-SYS-LL-09	test_co2_computation_based_on_accumulated_time	Verify that Timer and CO2 modules interact correctly to compute CO2 avoided from AutoStop duration	PASS
FT-SYS-LL-10	test_co2_reset_when_ignition_is_off	Verify that ignition OFF resets accumulated CO2 values across modules	PASS
FT-SYS-LL-11	test_fuel_computation_enabled_when_autostop_is_active	Verify that AutoStop state enables fuel saved accumulation and computation	PASS
FT-SYS-LL-12	test_fuel_computation_disabled_when_autostop_is_inactive	Verify that fuel accumulation/display logic is disabled when AutoStop becomes inactive	PASS
FT-SYS-LL-13	test_fuel_computation_based_on_accumulated_time	Verify that Timer and Fuel modules interact correctly to compute fuel saved from AutoStop duration	PASS
FT-SYS-LL-14	test_fuel_reset_when_ignition_is_off	Verify that ignition OFF resets accumulated fuel saved values across modules	PASS
FT-SYS-LL-15	test_restart_is_blocked_when_safe_stop_is_active	Verify that SafeStop generated by inhibit logic is correctly consumed by the Operation Module and prevents engine restart	PASS
FT-SYS-LL-16	test_restart_is_allowed_again_when_safe_stop_is_cleared	Verify that restart blocking is removed after SafeStop becomes inactive and a valid restart trigger is applied	PASS
FT-SYS-LL-17	test_autostop_state_is_maintained_while_safe_stop_is_active	Verify that the Operation Module remains in AutoStop state while SafeStop prevents restart	PASS
FT-SYS-LL-18	test_autostop_is_blocked_before_drive_cycle_memory_condition_is_met	Verify that AutoStop remains blocked before the drive cycle memory condition has been satisfied	PASS
FT-SYS-LL-19	test_autostop_is_allowed_after_drive_cycle_memory_condition_is_met	Verify that AutoStop becomes allowed after the drive cycle memory condition has been satisfied and the vehicle returns to standstill	PASS

6 Test Results

This section presents the results obtained from the execution of all test levels, including unit, integration, and functional tests, as well as CI workflow validation.

6.1 Unity Tests

The results of the unit tests are summarized in Figures 2 and 3.

Figure 2 presents the coverage summary for all modules, including statement, branch, and MC/DC metrics. As shown, all modules achieved coverage values above the defined threshold of 80%, with most modules reaching 100% in statement and branch coverage. Additionally, MC/DC coverage is fully achieved for the critical modules, demonstrating that all decision conditions were properly exercised.

Figure 3 presents the mutation testing results. The data indicate that the majority of generated mutants were detected by the test suite, achieving a high mutation score across all modules. This confirms that the unit tests are effective in identifying faults and validating the implemented logic.

Overall, both coverage and mutation testing results demonstrate that the unit test suite is comprehensive and satisfies the required quality criteria.

Coverage Summary

PASS — all modules meet threshold (80%)

Module	Statement	Branch	MC/DC	Status
co2	100.00% (34/34)	100.00% (4/4)	N/A	**PASS**
ss_control	92.54% (62/67)	93.33% (28/30)	100.00% (4/4)	**PASS**
ss_fuel	100.00% (29/29)	100.00% (4/4)	N/A	**PASS**
ss_inhibit_safety_conditions	100.00% (40/40)	95.24% (40/42)	89.47% (17/19)	**PASS**
ss_operation	97.01% (130/134)	100.00% (90/90)	100.00% (37/37)	**PASS**
timer	100.00% (26/26)	100.00% (4/4)	N/A	**PASS**

Figure 2: Coverage Summary results for all modules

```
[info] Mutation score: 92%
[info] Surviving mutants: 1
[info] Total execution time: 27ms
--- Mutation: build/mutation/test_ss_operation (src/ss_operation.c) ---
[info] Using config /workspaces/Start-Stop-System/build/mutation/run_ss_operation/mull.yml
[warning] Could not find dynamic library: libc.so.6
[info] Warm up run (threads: 1)
       [################################] 1/1. Finished in 1ms
[info] Applying file path filter (threads: 12)
       [################################] 96/96. Finished in 3ms
[info] Baseline run (threads: 1)
       [################################] 1/1. Finished in 1ms
[info] Running mutants (threads: 12)
       [################################] 33/33. Finished in 124ms
[info] Survived mutants (1/33):
/workspaces/Start-Stop-System/src/ss_operation.c:48:48: warning: Survived: Replaced <= with < [cxx_le_to_lt]
        return (in != NULL) && (in->vehicle_speed_kmh <= SS_AUTOSTOP_SPEED_MAX_KMH);
                                                      ^
[info] Mutation score: 96%
[info] Surviving mutants: 1
[info] Total execution time: 138ms
--- Mutation: build/mutation/test_timer (src/timer.c) ---
[info] Using config /workspaces/Start-Stop-System/build/mutation/run_timer/mull.yml
[warning] Could not find dynamic library: libc.so.6
[info] Warm up run (threads: 1)
       [################################] 1/1. Finished in 10ms
[info] Applying file path filter (threads: 12)
       [################################] 96/96. Finished in 2ms
[info] Baseline run (threads: 1)
       [################################] 1/1. Finished in 11ms
[info] Running mutants (threads: 5)
       [################################] 5/5. Finished in 15ms
[info] Mutation score: 100%
[info] All mutations have been killed
[info] Total execution time: 42ms

PASS — all modules meet threshold (80%)
```

Figure 3: Mutation Summary results for all modules

6.2 Integration Tests

The results of the integration tests are presented in Figure 4.

All defined integration test cases were executed successfully, with a total of 7 tests performed and no failures observed. Each test validates a specific interaction between system modules, including AutoStop activation, restart behavior, safety conditions, and estimation module responses.

The results confirm that the modules operate correctly when integrated, and that the overall system behavior aligns with the expected System Low-Level requirements. In particular, interactions involving operation logic, inhibit conditions, and estimation modules (CO_2 and Fuel) were verified without inconsistencies.

These results demonstrate that the integration between modules is stable and that the system behaves as expected under the tested scenarios.

```
ubuntu@43f86681dc58:/workspaces/Start-Stop-System$ make integration
clang-18 -std=c99 -Wall -Wextra -Wpedantic -Werror -g -O0 -I include/ -Itests/vendor/unity/src \
        tests/integration/test_integration_sys_ll.c src/co2.c src/ss_app.c src/ss_control.c src/ss_fuel.c src/ss_inhibit
build/test_integration_sys_ll
tests/integration/test_integration_sys_ll.c:491:test_integration_autostop_enables_estimation_modules:PASS
tests/integration/test_integration_sys_ll.c:492:test_integration_restart_disables_autostop_estimations:PASS
tests/integration/test_integration_sys_ll.c:493:test_integration_ignition_off_resets_estimation_outputs:PASS
tests/integration/test_integration_sys_ll.c:494:test_integration_safe_stop_blocks_restart:PASS
tests/integration/test_integration_sys_ll.c:495:test_integration_drive_cycle_memory_controls_autostop_permission:PASS
tests/integration/test_integration_sys_ll.c:496:test_integration_ss_enable_toggle_affects_operation_behavior:PASS
tests/integration/test_integration_sys_ll.c:497:test_integration_inhibit_conditions_block_autostop:PASS

-----------------------
7 Tests 0 Failures 0 Ignored
OK
```

Figure 4: Integration Test Results

6.3 Functional Tests

By evaluating the model/code under various simulated operational scenarios, it was possible to ensure the correct execution and interaction of the control logic modules. As depicted in Figure 5, the consolidated results illustrate the successful execution of the comprehensive test suite, thereby confirming functional compliance of the implemented architecture with project's requirements.

```
=================================================================
                        VERIFICATION REPORT
=================================================================
    Test_ID              Description                Checkpoint    Status
    _______    ___________________________________   __________    ______

    "LL-01"    "AutoStop Entry (Drive Cycle)"          8.5        "PASS"
    "LL-02"    "Engine Restart (Brake Release)"       18.5        "PASS"
    "LL-03"    "Button Noise Rejection"               28.5        "PASS"
    "LL-04"    "Button Abort Entry"                   38.5        "PASS"
    "LL-05"    "Toggle System Disable"                48.5        "PASS"
    "LL-06"    "Incline Inhibition (>=15%)"           58.5        "PASS"
    "LL-07"    "CO2 Accumulation Active"              68.5        "PASS"
    "LL-08"    "CO2 Accumulation Hold"                78.5        "PASS"
    "LL-09"    "CO2 Proportionality"                  88.5        "PASS"
    "LL-10"    "CO2 Reset (Ignition Off)"             98.5        "PASS"
    "LL-11"    "Fuel Metric Active"                  108.5        "PASS"
    "LL-12"    "Fuel Metric Hold"                    118.5        "PASS"
    "LL-13"    "Fuel Metric Proportionality"         128.5        "PASS"
    "LL-14"    "Fuel Reset (Ignition Off)"           138.5        "PASS"
    "LL-15"    "SafeStop (Door Open in AS)"          148.5        "PASS"
    "LL-16"    "SafeStop Recovery"                   158.5        "PASS"
    "LL-17"    "Door Open Inhibit"                   168.5        "PASS"
    "LL-18"    "Cycle Blocked (<8km/h)"              178.5        "PASS"
    "LL-19"    "System Re-validation"                188.5        "PASS"

-----------------------------------------------------------------
  SIMULATION TIME: 192.33 seconds
  TESTS EXECUTED:  19
  SUCCESS RATE:    100.0% (19/19)
=================================================================
>>
```

Figure 5: Test suite results - Simulink

6.4 CI workflow evidence

The results of the Continuous Integration (CI) workflow are presented in Figure 6.

All CI checks were successfully executed, with no failures observed. The workflow includes automated steps for build validation, formatting checks, static analysis (MISRA compliance), unit testing, coverage analysis, and mutation testing.

This workflow is automatically triggered for every Pull Request (PR), ensuring that all changes are validated before integration into the main branch. Each stage acts as a quality gate, verifying that the code meets the defined requirements for correctness, coverage thresholds, coding standards, and robustness.

The successful execution of all CI checks confirms that the system satisfies the required quality criteria and that the testing pipeline is fully operational and reliable.

Figure 6: CI workflows Summary

7 Appendix

The complete set of test cases developed for this project is available in the following spreadsheet:

- **Test Cases and Traceability:**
  `<https://docs.google.com/spreadsheets/d/1PdqRRzYBHCJ0tb6e9vSiisdIVNVEFBRR/edit?usp=sharing&ouid=102122488234889050998&rtpof=true&sd=true>`

This spreadsheet contains all unit, integration, and functional test cases, including their descriptions and traceability to the corresponding requirements.